

Linux File Permissions & System Administration Lab

A Hands-On Technical Guide

Platform: Kali Linux (Oracle VirtualBox)

User: phour, Layla (added)

Author: Adeolu Olatundun Ogunbiyi (Cybersecurity Analyst)

Date: March, 2026

Lab Steps: 24 | Commands: chmod, chown, chgrp, adduser, addgroup, sudo

This document provides a complete step-by-step walkthrough of Linux file permissions, ownership, user management, and privilege escalation concepts.

Every command shown was executed live and captured via terminal screenshot.

1. Lab Overview

This lab was conducted on Kali Linux running inside Oracle VirtualBox. All commands were executed in a Bash terminal as user phour, with sudo used where root-level privilege was required. The working directory throughout the main lab session is ~/Videos, which was used to create a representative file system structure.

The lab is structured as a **progressive workflow**: starting from directory creation, moving through every form of permission manipulation, transferring file ownership, creating users and groups, and finishing with a real-world privilege escalation scenario using the newly created user **Layla**.

Lab Environment

Component	Detail
Operating System	Kali Linux 2024 (GNOME / Wayland)
Virtualisation	Oracle VirtualBox
Shell	Bash
Primary User	phour
Secondary User	Layla (created during lab)
Working Directory	~/Videos
Files Used	movieslist, accesslist.txt
Groups Created	NewBorn

2. Linux Permissions Theory

Understanding Linux permissions is foundational to any security discipline. Every file and directory on a Linux system has three independent permission classes and two notation systems for expressing them.

2.1 The Three Permission Classes

Linux applies the principle of least privilege at the file level by dividing access control into three classes:

User (u): The owning user of the file. Has the most control. `chmod`, `chown`, and other privilege tools all operate relative to the owner.

Group (g): A named collection of users assigned to the file. Any member of the group interacts with the file under the group permission class.

Others (o): Every user who is neither the owner nor a member of the assigned group. This class represents the broadest access tier and should always carry the most restrictive permissions in a secure environment.

2.2 Symbolic Notation

Symbolic notation is the human-readable permission string shown by **ls -l**. It is ten characters long:

```
- r w x r w - r - -
```

```
| _____
```

```
| User  Group Others
```

```
└─ File type: - = regular file, d = directory, l = symlink
```

Each permission position uses one of four characters:

r Read. Can view file content (files) or list contents (directories). **w Write.**

Can modify file content (files) or create/delete within (directories).

x Execute. Can run the file as a program (files) or traverse the directory (directories).

- None. Permission not granted for this position.

Example from this lab:

```
-rwxrw---x 1 Layla NewBorn 0 Mar 31 movieslist
```

```
-  rwx rw- --x
```

User: rwx = read, write, execute

Group: rw- = read, write, no execute

Others: --x = no read, no write, execute only

Owner: Layla | **Group:** NewBorn

2.3 Octal (Numeric) Notation

Octal notation expresses each permission class as a single digit - the sum of the numeric values of granted permissions. It is deterministic: it sets the absolute permission state rather than adding or removing from an existing state.

Numeric values:

r = 4 (read)

w = 2 (write)

x = 1 (execute)

- = 0 (none)

Each class digit is the sum of its granted permissions.

For example:

$rw\text{x} = 4+2+1 = 7$,

$rw\text{-} = 4+2 = 6$, $r\text{-}\text{-}$

$= 4$, $\text{-}\text{-}\text{x} = 1$,

$\text{-}\text{-}\text{-} = 0$.

2.4 Octal Reference Table

Symbolic	Octal	Meaning
<code>-rwxrwxrwx</code>	<code>0777</code>	Full access for all classes — maximum risk
<code>-rwxrw---x</code>	<code>0761</code>	User full, Group read+write, Others execute only
<code>-rw-rw-rw-</code>	<code>0666</code>	Read+write for all, no execute
<code>-rwxr--r--</code>	<code>0744</code>	Owner full, Group and Others read only
<code>-rw-r--r--</code>	<code>0644</code>	Standard file: Owner rw, others read only
<code>-rw-rw-r--</code>	<code>0664</code>	Owner+Group rw, Others read only
<code>-rwx-----</code>	<code>0700</code>	Owner full, Group and Others no access
<code>-rw-----</code>	<code>0600</code>	Private file: Owner rw only
<code>---x--x--x</code>	<code>0111</code>	Execute only for all classes
<code>-----</code>	<code>0000</code>	No permissions for anyone

2.5 chmod Syntax Quick Reference

Symbolic format:

```
chmod [who][operator][permission] filename
```

```
who:    u (user) g (group) o (others) a (all)
```

```
operator: + (add) - (remove) = (set exact)
```

```
perm:    r (read) w (write) x (execute)
```

Octal format:

```
chmod NNN filename    # NNN = three-digit octal
```

```
chmod 755 /usr/local/bin/script
```

```
chmod -R 644 /var/www/html # recursive
```

3. Step-by-Step Lab Walkthrough

The following 24 steps were executed in order on the Kali Linux terminal. Each step shows the exact command used, the terminal screenshot as visual proof, and a technical explanation of what changed and why it matters.

Step 01 Making Directories and Listing the File System

Commands executed:

```
ls
pwd cd
Videos
mkdir zeeworld bollywood korean nollywood hollywood ls
mkdir Horror Comedy ls
```

The terminal screenshot shows a user named 'phour' on a Kali Linux machine. The session starts in the home directory (~). The user runs 'ls' to list existing directories: Desktop, Music, Public, zphisher, Documents, phishing_pot, Templates, Downloads, Pictures, and Videos. Then, the user changes to the Videos directory using 'cd Videos'. Next, the user runs 'mkdir zeeworld bollywood korean nollywood hollywood' to create five new directories. A subsequent 'ls' command shows these five directories. Finally, the user runs 'mkdir Horror Comedy' to create two more directories. A final 'ls' command confirms all seven directories (zeeworld, bollywood, korean, nollywood, hollywood, Horror, and Comedy) are present in the ~/Videos directory.

```
(phour@kali)-[~]
└─$ ls
Desktop  Music    Public  zphisher
Documents phishing_pot Templates
Downloads Pictures  Videos

(phour@kali)-[~]
└─$ pwd
/home/phour

(phour@kali)-[~]
└─$ cd Videos

(phour@kali)-[~/Videos]
└─$ mkdir zeeworld bollywood korean nollywood hollywood

(phour@kali)-[~/Videos]
└─$ ls
bollywood  korean  zeeworld
hollywood  nollywood

(phour@kali)-[~/Videos]
└─$ mkdir Horror Comedy

(phour@kali)-[~/Videos]
└─$ ls
bollywood  hollywood  korean  zeeworld
Comedy     Horror     nollywood
```

Fig 1 - Building a directory tree inside ~/Videos using mkdir

The session begins in the **home directory (~)**. **ls** confirms existing directories (**Desktop, Downloads, Music, Pictures, Videos**, etc.). After changing into **Videos**, a single **mkdir** call creates five directories simultaneously: **zeeworld, bollywood, korean, nollywood**, and **hollywood**. A second **mkdir** adds **Horror** and **Comedy**. The final **ls** confirms all seven directories exist.

Security note: New directories created without explicit permissions inherit from the process umask. Default umask 022 produces drwxr-xr-x (755), which allows any user to read and traverse but not write into them. A umask of 000 would produce 777 - a critical misconfiguration.

Step 02 Creating a File Using the touch Command

Command executed:

```
touch movieslist
```



```
(phour@kali)-[~/Videos]
$ touch movieslist

(phour@kali)-[~/Videos]
$ ls
bollywood  hollywood  korean      nollywood
Comedy      Horror      movieslist  zeeworld

(phour@kali)-[~/Videos]
$ cd Comedy

(phour@kali)-[~/Videos/Comedy]
$ ls

(phour@kali)-[~/Videos/Comedy]
$ pwd
/home/phour/Videos/Comedy

(phour@kali)-[~/Videos/Comedy]
$ cd

(phour@kali)-[~]
$ pwd
/home/phour
```

Fig 2 - touch creates an empty file named movieslist in ~/Videos

touch creates the file **movieslist** with zero bytes and the current timestamp. No content is written. The default permissions assigned by the system are controlled by the current umask. With umask 022, the result is -rw-r--r-- (0644): the owner can read and write, while group and others can only read.

Step 03 Changing Directory to Videos/Horror

Command executed:

```
cd Videos/Horror
```



```
phour@kali: ~/Videos

(phour@kali)-[~]
$ cd Videos

(phour@kali)-[~/Videos]
$ ls
bollywood  hollywood  korean  nollywood
Comedy     Horror     movieslist  zeeworld

(phour@kali)-[~/Videos]
$ cd Horror

(phour@kali)-[~/Videos/Horror]
$ pwd
/home/phour/Videos/Horror

(phour@kali)-[~/Videos/Horror]
$ ls

(phour@kali)-[~/Videos/Horror]
$ cd ..

(phour@kali)-[~/Videos]
$ pwd
/home/phour/Videos
```

Fig 3 - Navigating into the Horror subdirectory inside Videos

cd moves the shell's active path to the specified directory. The terminal prompt updates to reflect the new location: **(phour@kali)-[~/Videos/Horror]**. Traversing a directory requires execute (x) permission on it - not read. This distinction is important: a directory with r-- allows listing but not entering.

Step 04 Changing Directory to Korean

Command executed:

```
cd Korean
```

```
phour@kali: ~/Videos
(phour@kali)~[~/Videos]
$ ls
bollywood  hollywood  korean  nollywood
Comedy     Horror     movieslist  zeeworld

(phour@kali)~[~/Videos]
$ cd Korean
cd: no such file or directory: Korean

(phour@kali)~[~/Videos]
$ pwd
/home/phour/Videos

(phour@kali)~[~/Videos]
$ cd korean

(phour@kali)~[~/Videos/korean]
$ pwd
/home/phour/Videos/korean

(phour@kali)~[~/Videos/korean]
$ cd ~

(phour@kali)~[~]
$ pwd
/home/phour
```

Fig 4 - Navigating into the Korean subdirectory

Demonstrates relative directory navigation. From inside `~/Videos`, `cd Korean` resolves to `~/Videos/Korean`. This reinforces the concept that the execute bit on a directory controls traversal rights, not the read bit.

Step 05 Listing Directory Contents with Permissions, Users, and Groups

Commands executed:

```
cd Videos
ls ls
-l
```

```

phour@kali: ~/Videos
(phour@kali)-[~]
$ cd Videos
(phour@kali)-[~/Videos]
$ ls
bollywood  hollywood  korean  nollywood
Comedy      Horror      movieslist  zeeworld

(phour@kali)-[~/Videos]
$ ls -l
total 28
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 bollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 Comedy
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 hollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 Horror
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 korean
-rw-rw-r-- 1 phour phour 0 Mar 31 05:08 movieslist
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 nollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 zeeworld

```

Fig 5 - `ls -l` output showing permission strings, owners, groups, and timestamps

`ls -l` is the primary permission audit tool. The output shows every item in the `~/Videos` directory. Directories are marked with `d` and show `drwxrwxr-x`. The file `movieslist` shows `-rw-rw-r--` with owner `phour` and group `phour`.

Reading the `movieslist` row:

```
-rw-rw-r-- 1 phour phour 0 Mar 31 05:08 movieslist
```

	_____		_____	_____		_____	_____
	Perms		Links	Owner		Group	Size Timestamp Name
	- rw- rw- r--						
			_____	_____			

	User	Group	Others
—			

—	regular file
---	--------------

Step 06 Adding Write Permission to Others

Command executed:

```
chmod o+w Comedy
```

```

phour@kali: ~/Videos
(phour@kali)-[~/Videos]
$ chmod o+w Comedy
(phour@kali)-[~/Videos]
$ ls -l
total 28
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 bollywood
drwxrwxrwx 2 phour phour 4096 Mar 31 05:07 Comedy
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 hollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 Horror
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 korean
-rw-rw-r-- 1 phour phour 0 Mar 31 05:08 movieslist
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 nollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 zeeworld

```

Fig 6 - `chmod o+w Comedy` — `Comedy` changes from `drwxrwxr-x` to `drwxrwxrwx`

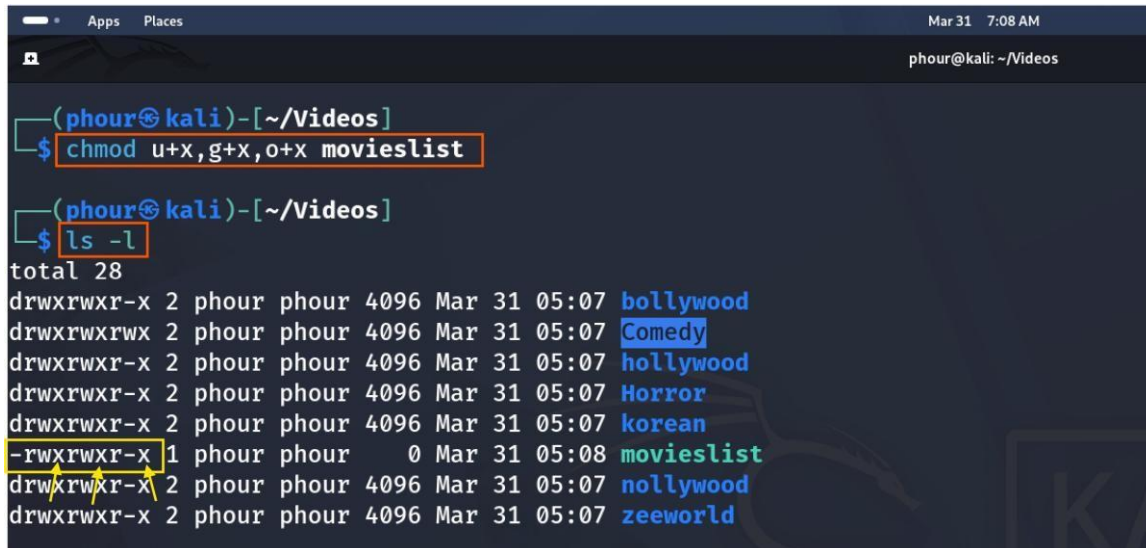
The **o+w** operator adds write permission to the others class for the `Comedy` directory.
Before: `drwxrwxr-x` (754-style). **After:** `drwxrwxrwx` (777). This means any user on the system can now create, delete, or rename files inside `Comedy`.

Security implication: World-writable directories are one of the most dangerous misconfigurations in a multi-user environment. An attacker with any shell access can place malicious files inside them, perform symlink attacks, or cause denial-of-service through disk space exhaustion.

Step 07 Adding Execute Permission to All Users, Groups, and Others

Command executed:

```
chmod u+x,g+x,o+x movieslist
```



```
(phour@kali)-[~/Videos]
$ chmod u+x,g+x,o+x movieslist

(phour@kali)-[~/Videos]
$ ls -l
total 28
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 bollywood
drwxrwxrwx 2 phour phour 4096 Mar 31 05:07 Comedy
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 hollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 Horror
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 korean
-rwxrwxr-x 1 phour phour 0 Mar 31 05:08 movieslist
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 nollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 zeeworld
```

Fig 7 - Execute permission added to all three classes for movieslist

The **comma-separated** syntax allows multiple class+permission pairs in a single chmod call. **u+x** adds execute for the owner, **g+x** for the group, and **o+x** for others. `movieslist` changes from `-rw-rw-r--` to `-rwxrwxr-x`. The others class gains x but still has no w, so they can execute but not modify the file.

For regular data files, execute permission is rarely appropriate and is a common indicator of a misconfiguration or a deliberate backdoor placement.

Step 08 Removing Read and Execute Permission from User, Group, and Others

Command executed:

```
chmod u-rx,g-rx,o-rx movieslist
```

```

(phour@kali)-[~/Videos]
$ chmod u-rx,g-rx,o-rx movieslist

(phour@kali)-[~/Videos]
$ ls -l
total 28
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 bollywood
drwxrwxrwx 2 phour phour 4096 Mar 31 05:07 Comedy
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 hollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 Horror
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 korean
--w--w---- 1 phour phour    0 Mar 31 05:08 movieslist
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 nollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 zeeworld

```

Fig 8 - All read and execute permissions stripped; only write remains

After this command, **movieslist** permissions become **--w--w----**. Only write permission remains for **user** and **group**. Others have nothing. This creates a paradox: the file can be overwritten but not read - not even by the owner. In practice, this state renders the file effectively inaccessible for normal use.

This is an important demonstration: a file with **w** but without **r** can receive new data blindly (e.g., output redirection) but its content cannot be verified by the user who wrote it, unless **r** is restored.

Step 09 Restoring Read Permission Using a+r

Command executed:

```
chmod a+r movieslist
```

```

phour@kali: ~/Videos
$ chmod a+r movieslist

phour@kali: ~/Videos
$ ls -l
total 28
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 bollywood
drwxrwxrwx 2 phour phour 4096 Mar 31 05:07 Comedy
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 hollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 Horror
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 korean
-rw-rw-r-- 1 phour phour 0 Mar 31 05:08 movieslist
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 nollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 zeeworld

```

Fig 9 - chmod a+r restores read for all three classes simultaneously

The shorthand **a** applies to all **three** classes at once (**equivalent to ugo**). **a+r** adds read for user, group, and others in a single operation. `movieslist` returns to **-rw-rw-r--** (read+write for user and group, read-only for others).

This demonstrates symbolic notation's additive property: **a+r** does not touch existing write or execute permissions. It only adds **r** where it was missing.

Step 10 Setting Execute-Only Permission Using Numeric Format (chmod 111)

Command executed:

```
chmod 111 movieslist
```



```

phour@kali: ~/Videos
$ chmod 111 movieslist

phour@kali: ~/Videos
$ ls -l
total 28
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 bollywood
drwxrwxrwx 2 phour phour 4096 Mar 31 05:07 Comedy
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 hollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 Horror
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 korean
---x--x--x 1 phour phour 0 Mar 31 05:08 movieslist
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 nollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 zeeworld

```

Fig 10 - `chmod 111`: execute-only for all classes (`---x--x--x`)

1 = execute only (x bit = 1, no r or w). Applied to all three classes. **movieslist** becomes `---x--x--x`. The file can be executed as a program by anyone, but no one - including the owner - can read or write its content.

This demonstrates the absolute-setting behaviour of octal notation: previous permissions are completely overwritten, not modified.

Octal breakdown: 1 1 1

```

| | └─ Others: --x
| └─── Group: --x
└────── User:  --x

```

Step 11 Setting Read+Write Permission Using Numeric Format (`chmod 666`)

Command executed:

```
chmod 666 movieslist
```



```

(phour@kali)-[~/Videos]
$ chmod 666 movieslist

(phour@kali)-[~/Videos]
$ ls -l
total 28
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 bollywood
drwxrwxrwx 2 phour phour 4096 Mar 31 05:07 Comedy
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 hollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 Horror
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 korean
-rw-rw-rw- 1 phour phour 0 Mar 31 05:08 movieslist
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 nollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 zeeworld

```

Fig 11 - `chmod 666`: read and write for all classes, no execute (-rw-rw-rw-)

6 = 4+2 = read+write. Applied identically across user, group, and others. **movieslist** becomes **-rw-rw-rw-**. Every user on the system can read and modify the file, but no one can execute it.

Security implication: **chmod 666** on any configuration file, credential file, or script is a critical vulnerability. Any local user can overwrite the file, potentially injecting malicious commands or stealing sensitive data. This setting is found in many privilege escalation CTF challenges.

Step 12 Setting Mixed Permissions Using Numeric Format (chmod 761)

Command executed:

```
chmod 761 movieslist
```

```

(phour@kali)-[~/Videos]
$ chmod 761 movieslist

(phour@kali)-[~/Videos]
$ ls -l
total 28
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 bollywood
drwxrwxrwx 2 phour phour 4096 Mar 31 05:07 Comedy
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 hollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 Horror
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 korean
-rwxrw---x 1 phour phour 0 Mar 31 05:08 movieslist
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 nollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 zeeworld

```

Fig 12 - `chmod 761`: User full access, Group read+write, Others execute only

761 demonstrates a mixed-permission configuration where each class has a different level of access. **movieslist** becomes **-rwxrw---x**.

Octal breakdown: 7 6 1

```

|  |  └─ Others: --x (1 = execute only)
|  └─── Group: rw- (4+2 = read+write)
└─── User:  rwx (4+2+1 = full)

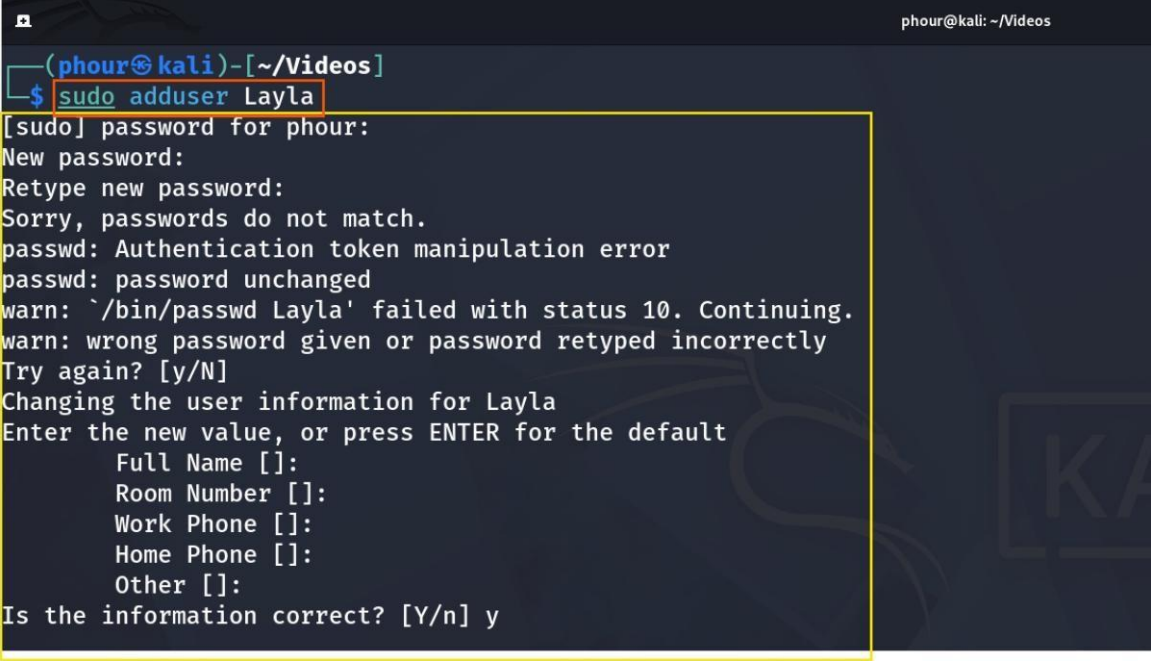
```

This permission state will persist through Steps 13-16, carried forward as ownership changes are applied.

Step 13 Creating a New User: Layla

Command executed:

```
sudo adduser Layla
```



```

(phour@kali)-[~/Videos]
$ sudo adduser Layla
[sudo] password for phour:
New password:
Retype new password:
Sorry, passwords do not match.
passwd: Authentication token manipulation error
passwd: password unchanged
warn: `/bin/passwd Layla' failed with status 10. Continuing.
warn: wrong password given or password retyped incorrectly
Try again? [y/N]
Changing the user information for Layla
Enter the new value, or press ENTER for the default
    Full Name []:
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] y

```

Fig 13 - sudo adduser Layla - interactive user creation with password prompt **adduser** is an interactive command that creates a new user account. The session shows the password prompt, a failed password match attempt, and then the user information form (Full Name, Room, Phone). The user confirms with 'y' and the account is created.

The adduser command creates:

- A new entry in /etc/passwd with the username, UID, home directory, and shell
- A home directory at /home/Layla with standard skeleton files
- A hashed password entry in /etc/shadow
- A primary group named Layla (same name as user, by default)

Security note: A newly created user has no sudo access by default. This is the correct, secure behaviour enforced by the absence of a sudoers entry. **Steps 23-24** demonstrate and correct this state.

Step 14 Changing File Owner to Layla

Commands executed:

```

chown Layla movieslist    # FAILS - Operation not permitted sudo
chown Layla movieslist    # SUCCESS with elevated privilege ls -l

```

```

(phour@kali)-[~/Videos]
$ ls
bollywood  hollywood  korean  nollywood
Comedy     Horror     movieslist  zeeworld

(phour@kali)-[~/Videos]
$ chown Layla movieslist
chown: changing ownership of 'movieslist': Operation not permitted

(phour@kali)-[~/Videos]
$ sudo chown Layla movieslist

(phour@kali)-[~/Videos]
$ ls -l
total 28
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 bollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 Comedy
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 hollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 Horror
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 korean
-rwxrw---x 1 Layla phour 0 Mar 31 05:08 movieslist
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 nollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 zeeworld

```

Fig 14 - *chown* without *sudo* is rejected; *sudo chown* succeeds. Owner changes to Layla.

The first **chown** attempt fails with '**Operation not permitted**' because only **root** can change file ownership on Linux. **This is a deliberate security control**: if any user could reassign ownership, they could transfer files away from restricted users or claim ownership of files they should not control.

With **sudo**, the command succeeds. **ls -l** confirms the owner has changed from **phour** to **Layla**. The group remains **phour** and the permissions stay **-rwxrw---x**.

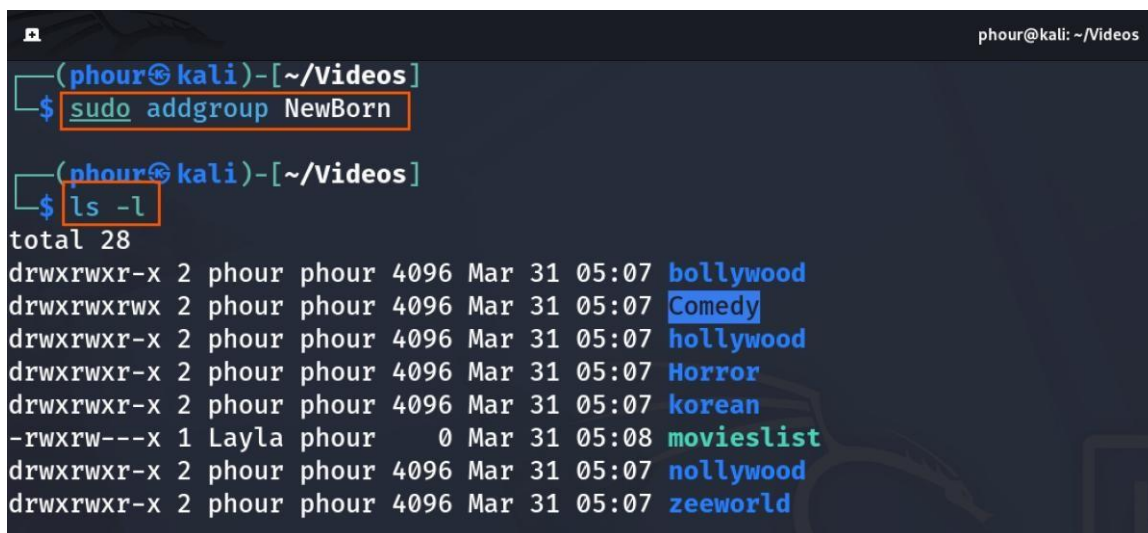
Before: -rwxrw---x 1 phour phour 0 Mar 31 05:08 movieslist

After: -rwxrw---x 1 Layla phour 0 Mar 31 05:08 movieslist

Step 15 Creating a New Group: NewBorn

Command executed:

```
sudo addgroup NewBorn
```



```

(phour@kali)-[~/Videos]
$ sudo addgroup NewBorn

(phour@kali)-[~/Videos]
$ ls -l
total 28
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 bollywood
drwxrwxrwx 2 phour phour 4096 Mar 31 05:07 Comedy
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 hollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 Horror
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 korean
-rwxrw---x 1 Layla phour 0 Mar 31 05:08 movieslist
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 nollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 zeeworld

```

Fig 15 - sudo addgroup NewBorn creates a new system group

addgroup creates a new group entry in **/etc/group** with an automatically assigned GID. The group **NewBorn** is now available for file assignment. At this point the group has no members; it simply exists as an access control identifier.

Verification: `grep NewBorn /etc/group` would show an entry like: `NewBorn:x:1002:`

Step 16 Changing Group Owner to NewBorn

Command executed:

```

sudo chgrp NewBorn movieslist ls
-l

```



```

(phour@kali)-[~/Videos]
$ sudo chgrp NewBorn movieslist

(phour@kali)-[~/Videos]
$ ls -l
total 28
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 bollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 Comedy
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 hollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 Horror
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 korean
-rwxrw---x 1 Layla NewBorn 0 Mar 31 05:08 movieslist
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 nollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 zeeworld

(phour@kali)-[~/Videos]
$ ls -l
total 28
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 bollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 Comedy
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 hollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 Horror
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 korean
-rwxrw---x 1 Layla NewBorn 0 Mar 31 05:08 movieslist
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 nollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 zeeworld

```

Fig 16 - `chgrp` transfers group ownership of `movieslist` from `phour` to `NewBorn` `chgrp` changes the group associated with the file without touching the owning user. After this command, `movieslist` is owned by `Layla` and assigned to group `NewBorn`. Any member of `NewBorn` now interacts with the file under the group permission class (`rw-`).

Before: `-rwxrw---x 1 Layla phour 0 Mar 31 05:08 movieslist`

After: `-rwxrw---x 1 Layla NewBorn 0 Mar 31 05:08 movieslist`

The final ownership state: Layla as the individual owner with full access (`rw-`), members of `NewBorn` with read+write (`rw-`), and all others with execute only (`--x`).

Step 17 Moving a File into a Directory

Commands executed:

```

mv movieslist Horror
ls -l
cd Horror

```

```
ls ls
-l
```

```
(phour@kali)-[~/Videos]
$ mv movieslist Horror

(phour@kali)-[~/Videos]
$ ls -l
total 28
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 bollywood
drwxrwxrwx 2 phour phour 4096 Mar 31 05:07 Comedy
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 hollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 06:29 Horror
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 korean
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 nollywood
drwxrwxr-x 2 phour phour 4096 Mar 31 05:07 zeeworld

(phour@kali)-[~/Videos]
$ cd Horror

(phour@kali)-[~/Videos/Horror]
$ ls
movieslist

(phour@kali)-[~/Videos/Horror]
$ ls -l
total 0
-rwxrw---x 1 Layla NewBorn 0 Mar 31 05:08 movieslist
```

Fig 17 - mv relocates movieslist into the Horror subdirectory

mv moves the file from `~/Videos` to `~/Videos/Horror`. After the move, **ls -l** in Videos no longer shows movieslist. After **cd Horror**, **ls** confirms the file is present, and **ls -l** shows it retains all permissions and ownership from before the move:

```
-rwxrw---x 1 Layla NewBorn 0 Mar 31 movieslist
```

mv does not reset permissions or ownership. This matters for security: if a file with world-writable permissions is moved into a restricted directory, it retains those permissions. The directory's permissions control who can see the file, but not the file's own access controls.

Step 18 Removing a File and Additional File Operations

Commands executed:

```
sudo rm movieslist # from ~/Videos/Horror
```

```
ls                # confirms Horror is empty cd
..
ls
touch accesslist.txt mv
accesslist.txt bollywood cd
bollywood
```

```
(phour@kali)-[~/Videos/Horror]
$ sudo rm movieslist
(phour@kali)-[~/Videos/Horror]
$ ls
(phour@kali)-[~/Videos/Horror]
$ cd ..
(phour@kali)-[~/Videos]
$ ls
bollywood  hollywood  korean  zeeworld
Comedy     Horror     nollywood
(phour@kali)-[~/Videos]
$ touch accesslist.txt
(phour@kali)-[~/Videos]
$ mv accesslist bollywood
mv: cannot stat 'accesslist': No such file or directory
(phour@kali)-[~/Videos]
$ mv accesslist.txt bollywood
(phour@kali)-[~/Videos]
$ cd bollywood
```

Fig 18 - `sudo rm` deletes `movieslist`; `accesslist.txt` is created and moved to `bollywood` **rm** requires **sudo** here because `movieslist` is owned by Layla, not phour. After deletion, the `Horror` directory is empty. A new file `accesslist.txt` is then created with **touch** and moved into the `bollywood` directory.

Note: the first **mv** attempt fails ('`mv accesslist bollywood`' - no such file) because the filename was not typed with the **.txt** extension.

rm deletes files permanently. There is no recycle bin. On production systems, accidental **rm** of important files requires restoration from backup or filesystem snapshot.

Step 19 Using rmdir, ls, and cd to Clean Up Directories

Commands executed:

```
cd bollywood
ls          # shows accesslist.txt
ls -l       # shows -rw-rw-r-- 1 phour phour 0 Mar 31 accesslist.txt
cd ..
ls
rmdir Horror # removes now-empty Horror directory ls
# Horror is gone
```

```
(phour@kali)~[/Videos]
$ cd bollywood
(phour@kali)~[/Videos/bollywood]
$ ls
accesslist.txt
(phour@kali)~[/Videos/bollywood]
$ ls -l
total 0
-rw-rw-r-- 1 phour phour 0 Mar 31 06:30 accesslist.txt
(phour@kali)~[/Videos/bollywood]
$ cd ..
(phour@kali)~[/Videos]
$ ls
bollywood  hollywood  korean     zeeworld
Comedy      Horror      nollywood

(phour@kali)~[/Videos]
$ rmdir Horror
(phour@kali)~[/Videos]
$ ls
bollywood  Comedy  hollywood  korean  nollywood  zeeworld
```

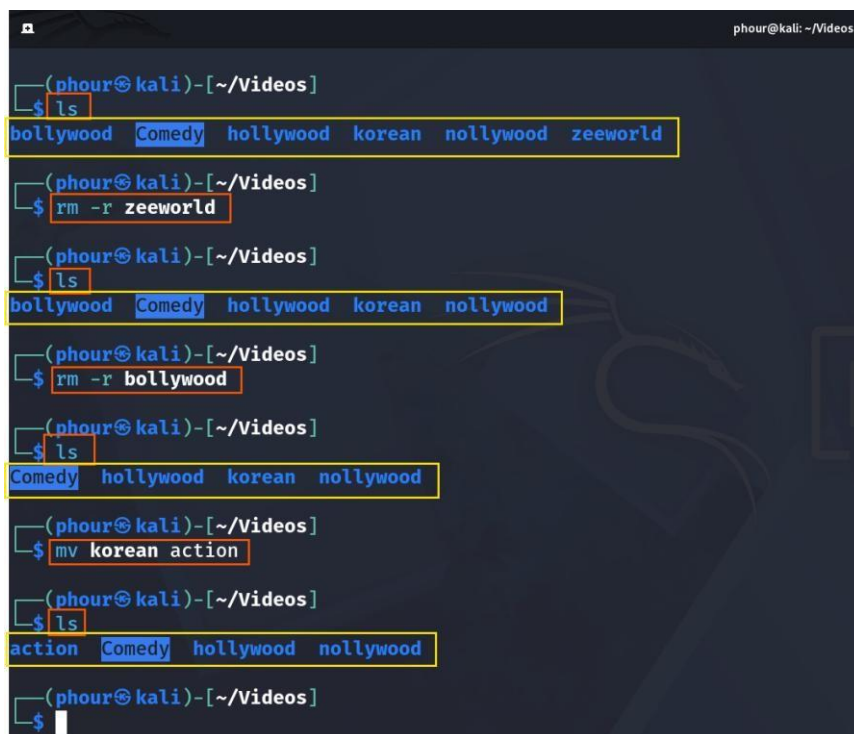
Fig 19 - `rmdir` removes the empty `Horror` directory; `accesslist.txt` verified in `bollywood` `rmdir` only operates on empty directories. **Horror** is now empty (`movieslist` was removed in Step 18), so `rmdir` succeeds. If `Horror` contained files, `rmdir` would fail with '**Directory not empty**' - this is a safety feature that prevents accidental recursive deletion.

`accesslist.txt` in `bollywood` shows `-rw-rw-r--` (0664), the default for `touch` with the current umask. Owner and group are `phour`.

Step 20 Removing Directories with `rm -r`

Commands executed:

```
rm -r zeeworld    # remove zeeworld and all contents ls
rm -r bollywood   # remove bollywood (contains accesslist.txt) ls
mv korean action  # rename korean to action ls
```



```
(phour@kali)~[/Videos]
$ ls
bollywood Comedy hollywood korean nollywood zeeworld

(phour@kali)~[/Videos]
$ rm -r zeeworld

(phour@kali)~[/Videos]
$ ls
bollywood Comedy hollywood korean nollywood

(phour@kali)~[/Videos]
$ rm -r bollywood

(phour@kali)~[/Videos]
$ ls
Comedy hollywood korean nollywood

(phour@kali)~[/Videos]
$ mv korean action

(phour@kali)~[/Videos]
$ ls
action Comedy hollywood nollywood

(phour@kali)~[/Videos]
$
```

Fig 20 - `rm -r` removes non-empty directories; `mv` renames a directory

rm -r performs recursive deletion; it removes the target directory and everything inside it without prompting for confirmation unless **-i** is also specified. **zeeworld** and **bollywood** (containing `accesslist.txt`) are both removed. **korean** is renamed to **action** using **mv**.

Security implication: **rm -rf** is one of the most destructive commands available. On production systems, this command without adequate privilege controls or filesystem-level protections can cause irreversible data loss. Always confirm the target path before executing.

Step 21 Signing into User Layla - Stage 1: Logging Out of phour

Action performed:

System tray → Power menu → Log Out

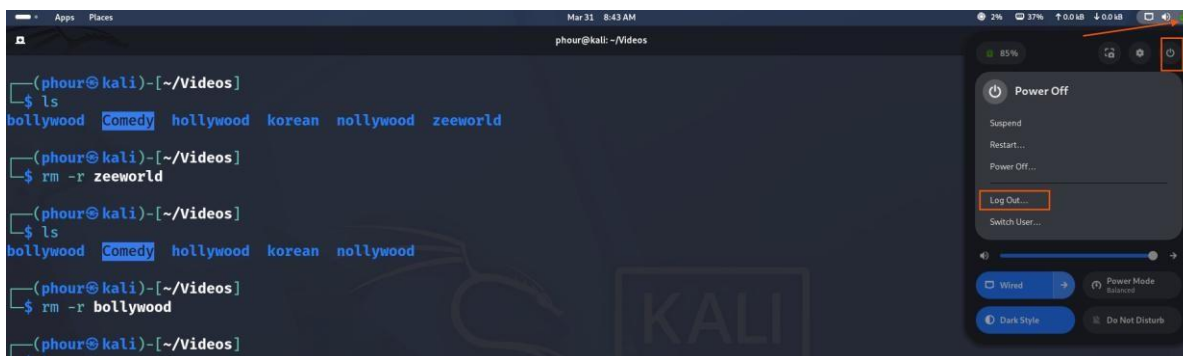


Fig 21 - Kali Linux desktop with power menu open; Log Out selected to switch user

To sign in as **Layla**, the **phour** session is first terminated via the GNOME **power menu**. **Log Out** ends the current desktop session and returns to the GDM (GNOME Display Manager) login screen, where user selection is available.

Step 22 Signing into User Layla - Stage 2: Kali Login Screen

Action performed:

Select 'Layla' from login screen → enter password

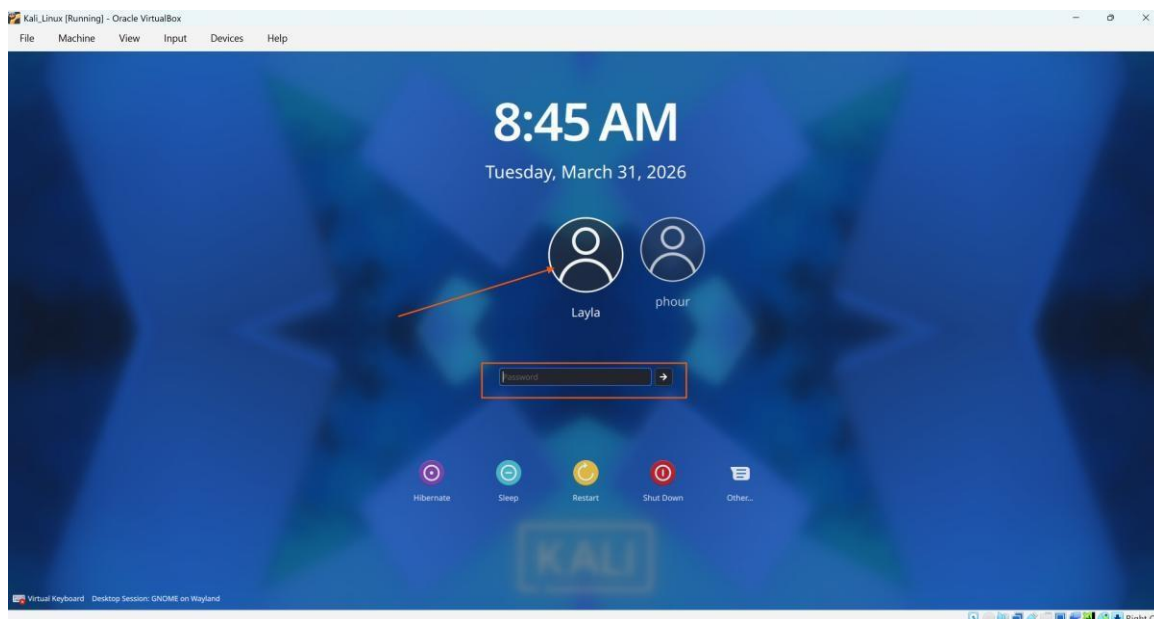


Fig 22 - GDM login screen showing both Layla and phour accounts; Layla is selected

The login screen confirms that **Layla's** account was successfully created in **Step 13**. Both accounts (**Layla** and **phour**) are visible. The arrow and password field indicate **Layla** is being selected for authentication.

This stage is critical for the privilege escalation demonstration: Layla's session is a **standard unprivileged** user session. She has no entry in `/etc/sudoers` and is not a member of the **sudo** group.

Step 23 User Without sudo Privilege - Failed Escalation Attempt

Command executed as Layla:

```
sudo adduser Cara
```

```

(Layla@kali)-[~]
$ sudo adduser Cara
[sudo] password for Layla:
Layla is not in the sudoers file.

(Layla@kali)-[~]
$

```

Fig 23 - Layla attempts a sudo command and is denied: 'not in the sudoers file'

Logged in as **Layla**, the terminal prompt shows **(Layla@kali)-[~]**. Attempting **sudo adduser Cara** prompts for Layla's password, then returns:

Layla is not in the sudoers file.

This is the expected, secure behaviour. The system checks **/etc/sudoers** and its include files (**/etc/sudoers.d/**) for an entry permitting **Layla** to run sudo. Finding none, the request is denied and the attempt is logged.

What gets logged: Every failed sudo attempt generates an entry in **/var/log/auth.log**. SOC analysts and system administrators monitor this log to detect **brute-force privilege escalation attempts**. The message **'This incident will be reported'** (shown in some distributions) is not a bluff; it is a confirmation that the event was written to the audit trail.

To verify from the admin account:

```
grep 'Layla' /var/log/auth.log
```

```
# Output: Mar 31 08:57:xx kali sudo: Layla : user NOT in sudoers ...
```

Step 24 Adding User Layla to the sudo Group

Commands executed (back as phour):

```
sudo adduser Layla sudo
sudo adduser Layla sudo # run twice to confirm idempotence
```

```

(phour@kali)-[~]
$ sudo adduser Layla sudo
[sudo] password for phour:
(phour@kali)-[~]
$ sudo adduser Layla sudo
warn: The user 'Layla' is already a member of 'sudo'.
(phour@kali)-[~]
$

```

Fig 24 - sudo adduser Layla sudo adds Layla to the sudo group; second run confirms membership

Back in phour's session, running **sudo adduser Layla sudo** adds **Layla** to the **sudo supplementary group**. The second run of the same command returns:

```
warn: The user 'Layla' is already a member of 'sudo'.
```

This confirms the operation was idempotent - running it twice does not create a duplicate entry or cause an error beyond a warning. After this change and a new login session, Layla can **execute privileged commands**.

How it works: The `/etc/sudoers` file on Kali (and most Debian-based systems) contains a rule granting all members of the **sudo group unrestricted root access**: `%sudo ALL=(ALL:ALL) ALL`. Adding Layla to the sudo group activates this rule for her account.

```
# /etc/sudoers (relevant line):
```

```
%sudo ALL=(ALL:ALL) ALL
```

```
#
```

```
# Group 'sudo' may run any command as any user on any host
```

4. Command Reference

The following table summarises every command used in this lab with its syntax, purpose, and privilege requirement.

Command	Syntax	Purpose	Root?
mkdir	mkdir [-p] dir	Create directories	No
touch	touch filename	Create empty file / update timestamp	No
cd	cd path	Change active directory	No
ls -l	ls [-la] [path]	List with permissions, owner, size	No
chmod	chmod [sym oct] file	Change file permissions	Owner/Root
chown	chown user[:grp] file	Change owning user	Root only
chgrp	chgrp group file	Change owning group	Root only
adduser	sudo adduser user	Create new user interactively	Root
addgroup	sudo addgroup group	Create new system group	Root
adduser (group)	sudo adduser u g	Add user to supplementary group	Root
mv	mv src dst	Move or rename file/directory	No
rm	rm [-rf] file	Delete file(s) permanently	Varies
rmdir	rmdir dir	Delete empty directory only	No
su	su [-] user	Switch active user in terminal	Target password
sudo	sudo command	Run single command as root	Sudoers

pwd	pwd	Print working directory path	No
-----	-----	------------------------------	----

5. Security Analysis

The lab deliberately demonstrates several common misconfigurations that create realworld vulnerabilities. Each is described below with its attack vector and remediation.

5.1 World-Writable Directories (chmod 777)

Step 6 applied **chmod o+w** to the **Comedy directory**, producing **drwxrwxrwx**. This configuration allows any user on the system to create, delete, or rename files within that directory.

Attack scenarios enabled by world-writable directories:

- **Symlink attack:** An attacker creates a symlink inside the directory pointing to **/etc/passwd**. If a privileged script follows the symlink, it may overwrite the target.
- **Cron job hijack:** If a root-owned cron job writes to a world-writable path, an attacker can pre-create a file with malicious content at that path.
- **Disk exhaustion:** An attacker can fill the directory with junk data, causing disk-full conditions that crash services.
- **Library injection:** In world-writable directories on the **LD_LIBRARY_PATH**, a malicious **.so** file can be injected and loaded by legitimate processes.

Remediation:

```
chmod o-w directory_name # remove write from others
```

```
chmod 1777 /tmp # sticky bit: only owner can delete their files
```

5.2 Permissive File Settings (chmod 666 and 777 on files)

Step 11 set **movieslist** to **666 (-rw-rw-rw-)**.

Steps 12-16 further show the file transitioning through various states. A **666** file is readable and writable by every user.

Real-world risk: configuration files, database credential files, private keys, and shell scripts with **666** permissions are among the most common findings in Linux privilege escalation CTFs and penetration tests. A readable **/etc/shadow** (even partial) immediately exposes password hashes. A writable **/etc/passwd** allows direct UID 0 user injection.

How to detect:

```
find / -perm -0002 -type f 2>/dev/null # world-writable files
```

```
find / -perm -0006 -type f 2>/dev/null # world-readable+writable
```

```
find / -perm -4000 -type f 2>/dev/null # SUID files (escalation vector)
```

5.3 Ownership Misconfiguration

Steps 14 and **16** transfer ownership to **Layla** and group to **NewBorn** respectively.

The critical insight: once ownership is transferred, the previous owner (phour) loses the ability to modify the file's permissions - unless they use sudo.

Attack scenario: If a system service runs as a specific user and its configuration file is accidentally chowned to a regular user, that user can modify the configuration to redirect service behaviour; potentially to run code under the service's identity.

5.4 Privilege Creep via sudo Group Membership

Step 24 adds **Layla** to the **sudo group**. On Debian/Kali systems, sudo group membership grants **unrestricted root access** via the rule: **%sudo ALL=(ALL:ALL) ALL**.

Privilege creep occurs when users are added to high-privilege groups for temporary reasons and never removed. Over time, the number of users with sudo access grows beyond operational need, expanding the attack surface for credential theft.

Best practices:

```
# Audit sudo group members:
```

```
grep '^sudo:' /etc/group
```

```
# Grant specific commands only (principle of least privilege):
```

```
# In /etc/sudoers.d/layla:
```

```
Layla ALL=(ALL) /usr/sbin/adduser, /usr/sbin/addgroup
```

```
# Revoke sudo access when no longer needed:
```

```
sudo deluser Layla sudo
```

5.5 Permissions vs. Ownership - Key Distinction

Many security practitioners confuse these two controls. They work together but address different questions:

Permissions (chmod)	Ownership (chown / chgrp)
What can be done with the file	Who can do what with the file
Read, write, execute flags for each class	Which user and group the file belongs to
Changed with chmod by owner or root	Changed with chown/chgrp by root only
Defines the capability boundaries	Defines which class a user falls into

6. Common Mistakes and How to Avoid Them

Using chmod 777 as a quick fix `chmod 777` on a file or directory resolves permission errors immediately; which is why

beginners reach for it. It also grants every local user full access. Always diagnose the actual permission needed and apply only that.

Forgetting -a flag with usermod -G `usermod -G sudo Layla` without **-a** REPLACES all supplementary groups. Layla would lose membership in every other group she belongs to. Always use `usermod -aG` (append) or `adduser user group`.

Running chown without sudo

Only root can change file ownership. Attempting `chown` as a regular user returns 'Operation not permitted' - as demonstrated in **Step 14**. Always prefix with `sudo`.

Confusing r on a directory with x

Read (r) on a directory allows listing its contents with `ls`. **Execute (x)** allows traversing into it with `cd`. You can have `r` without `x` (can see names but cannot enter) or `x` without `r` (can enter if you know the path but cannot list it).

Using rm -rf without verifying the path `rm -rf` is immediate and permanent. A typo in the path; especially as **root** - can delete critical system directories. Always run `ls` on the target path **first**.

Not reloading the session after group changes

Group membership changes take effect on the next login. Adding a user to `sudo` does not grant `sudo` access in the current terminal session. The user must log out and back in (or use `newgrp sudo` in some configurations).

8. Quick Reference Cheatsheet

Symbolic chmod

```
chmod u+x file      # add execute for owner
chmod g-w file      # remove write from group
chmod o=r file      # set others to read-only (exact)
chmod a+r file      # add read for all
chmod u+x,g-w file  # multiple changes at once
chmod -R 755 dir/   # recursive
```

Numeric chmod

```
chmod 777 file      # rwxrwxrwx — full access all (dangerous)
chmod 755 file      # rwxr-xr-x — standard executable chmod
644 file          # rw-r--r-- — standard file chmod 600 file      #
rw----- — private file chmod 666 file      # rw-rw-rw- — all
read+write (risky) chmod 111 file      # --x--x--x — execute
only all chmod 761 file      # rwxrw---x — mixed (this lab)
chmod 1777 dir/     # rwxrwxrwx + sticky bit (/tmp style)
```

Ownership

```
sudo chown Layla file      # change owner sudo chown
Layla:NewBorn file      # change owner and group sudo chgrp
NewBorn file          # change group only sudo chown -R
phour:phour dir/      # recursive ownership change User and
```

Group Management

```
sudo adduser username      # create user interactively sudo
addgroup groupname        # create group sudo adduser
username groupname      # add user to group sudo deluser
username groupname      # remove user from group groups
username                # list group memberships id username
# full UID/GID info cat /etc/group | grep groupname      # verify
group existence grep '^sudo:' /etc/group                # list all sudo
members
```

Audit Commands

```
ls -l path          # view permissions ls -la path
# include hidden files stat filename          # detailed
file metadata find / -perm -0002 -type f 2>/dev/null #
world-writable files find / -perm -4000 -type f 2>/dev/null #
SUID files grep 'sudo' /var/log/auth.log      # sudo audit trail
```

End of Document - Linux File Permissions Lab - Kali Linux - March, 2026

*Created by: Adeolu Olatundun Ogunbiyi
Cybersecurity Analyst*